Game Variant Fire Brigade

R.A. Diaz Lozada, A.M. Cepeda Villanueva, J.F. Cuervo Huertas
Systems Engineering, School of Engineering
Universidad Distrital Francisco José de Caldas
{radiazl, amcepedav, jfcuerpoh}@udistrital.edu.co
Bogotá, Colombia

***Abstract*—Forest fire management requires rapid decision-making, balancing fire suppression with civilian evacuation. This paper presents Fire Brigade, a turn-based simulation game on an 8×8 grid that demonstrates greedy and backtracking algorithms. The greedy dispatcher always sends units to the highest-risk cell, determined by a Binary Search Tree (BST). The backtracking evacuator finds safe evacuation routes avoiding burning cells. The game features three difficulty levels, three unit types, four civilian categories, and two fire propagation behaviors. This document presents the complete system design, including architecture, data structures, algorithms, and communication protocols.**

***Index Terms*—Greedy Algorithm, Backtracking, Binary Search Tree, Linked List, Turn-Based Simulation.**

I. Introduction

Forest fires represent one of the most challenging natural disasters to manage, requiring rapid decision-making under pressure and efficient allocation of limited resources. Firefighting operations must balance two critical objectives: extinguishing active fires to prevent further spread, and evacuating civilians from danger zones before they become trapped. These challenges make forest fire management an ideal domain for exploring algorithmic solutions, where computational strategies can support or even automate tactical decisions.

Fire Brigade is a turn-based simulation game that models this problem on an 8×8 grid. The player takes command of limited firefighting resources and must decide which units to deploy and which civilians to evacuate, while the algorithms determine the optimal locations and routes. The game is designed to demonstrate the practical application of two classic algorithmic strategies: a greedy algorithm for unit dispatch, which always selects the cell with the highest fire risk; and a backtracking algorithm for civilian evacuation, which exhaustively explores all possible safe paths to the grid boundary.

The project integrates three distinct technological layers: a C++ engine responsible for core data management and structural logic, a Python module housing the decision-making algorithms, and a Pygame-based graphical interface. Communication between the C++ engine and the Python layer is handled through a JSON bridge, allowing each component to operate independently while efficiently sharing game state.

The C++ engine manages two key data structures: a singly linked list that implements the unit dispatch queue, storing incoming firefighting units (ground crews, trucks, and helicopters) in order of arrival; and a Binary Search Tree (BST) that organizes burning cells by their computed risk score, enabling the greedy algorithm to retrieve the maximum-risk target in O(log n) time. Risk is calculated based on fire intensity and type, with canopy fires receiving a multiplier of 1.5 due to their more aggressive eight-directional propagation.

The game features three difficulty levels — Easy, Medium, and Hard — each varying the number and type of fires, the composition of civilians on the map, the initial unit allocation, and the rate at which new units arrive. Three built-in variants add strategic depth: a scarce resources variant governing unit types and arrival timing, a civilian priorities variant assigning different point values and movement speeds to four civilian categories (Child, Adult, Elderly, Family Group), and a dynamic fire behavior variant introducing two fire propagation modes with distinct spread patterns.

The development effort was distributed across three complementary roles, each aligned with the architectural layers:

- **Jeison Felipe Cuervo Huertas** — Engine Developer: implemented the C++ core, including the linked list dispatch queue, the BST risk-ranking structure, and the JSON bridge connecting the engine to the Python layer.
- **Andres Mauricio Cepeda Villanueva** — Algorithm Developer: implemented the Greedy dispatcher and the Backtracking evacuator in Python, including tie-breaking logic and path optimization.
- **Raul Andres Diaz Lozada** — UI Developer: built the Pygame interface, rendering the 8×8 grid, fire propagation animations, evacuation sequences, and the score and resource HUD.

II. Methods and Materials

A. System Architecture

The system comprises three independent layers:

- **C++ Engine:** Manages game state, fire propagation, unit queue (linked list), and BST for risk ranking.
- **Python Module:** Executes Greedy dispatcher or Backtracking evacuator based on player action.
- **Pygame Interface:** Renders grid, fires, civilians, HUD, and captures player input.

Communication occurs via JSON. The engine serializes state after each turn; Python returns decisions as JSON responses.

B. Data Structures

1) Linked List – Unit Dispatch Queue: FIFO queue implemented as singly linked list. Each node stores unit type and arrival turn.

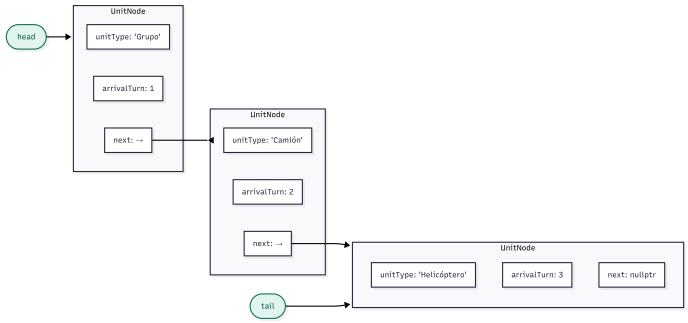

Fig. 1. Linked List Structure

2) Binary Search Tree – Risk Levels: BST organizes burning cells by risk value. Each node stores risk level, coordinates, fire type, and child pointers.

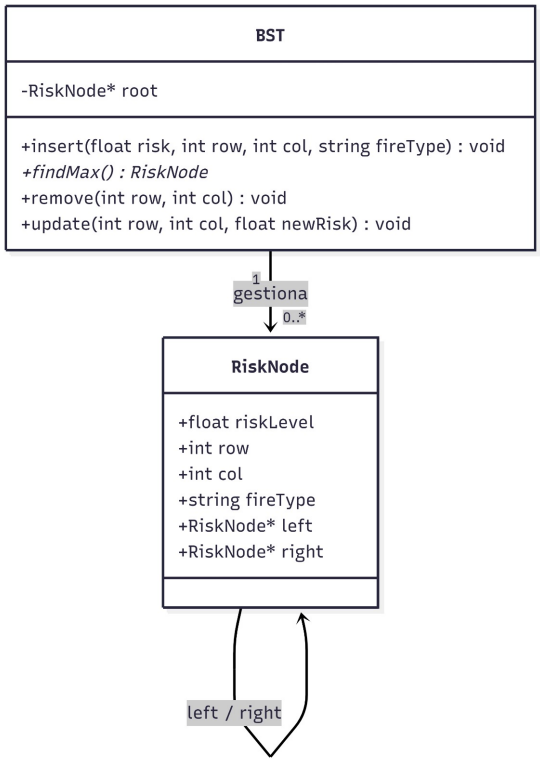

Fig. 2. BST Node Structure

Risk is calculated as: `Risk = intensity × (1.5 if canopy, else 1.0)`.

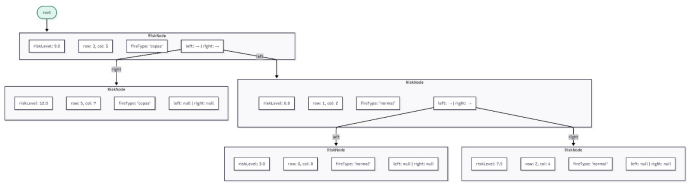

Fig. 3. Example BST Nodes

C. Algorithms

1) Greedy Dispatcher: Selects cell with highest risk from BST. Ties broken by: (1) canopy fire preference, (2) smallest coordinates.

[H] Greedy Dispatcher

Require: BST ordered by risk
Ensure: target cell (row, column)

```
target ← bst.getHighestRiskCell()
if multiple cells have same risk then
    if canopy fire exists then
        target ← select canopy fire
    else
        target ← select smallest row, then column
    end if
end if
return target
```

2) Backtracking Evacuator: Finds shortest safe route to border using depth-first search.

[H] Backtracking Evacuation

Require: start position, obstacles matrix
Ensure: route, success

```
visited ← False matrix
explore(start, visited, [])
if bestRoute found then
    return bestRoute, True
else
    return [], False
end if
```

[H] Explore (Recursive)

```
Append position to currentRoute
if position on border then
    update bestRoute; RETURN True
end if
visited[pos] ← True
for each movement in [(-1,0),(1,0),(0,-1),(0,1)] do
    if valid move then
        explore(newPos)
    end if
end for
visited[pos] ← False; remove last from route
```

D. Game Components

TABLE I
UNIT TYPES

Unit	Effect	Arrival (turns)
Crew	Extinguishes one cell	2–4
Truck	Extinguishes any cell	4–6
Helicopter	Extinguishes + protects neighbors	6–8

TABLE II
CIVILIAN VALUES

Type	Points
Child	100
Adult	50
Elderly	75
Family Group	150

TABLE III
FIRE TYPES

Type	Spread	Risk Mult.
Normal	4 directions	1.0
Canopy	8 directions	1.5

TABLE IV
DIFFICULTY LEVELS

Parameter	Easy	Medium	Hard
Initial Fires	2 normal	2 normal + 1 canopy	2 normal + 3 canopy
Initial Civilians	Adult, Child	+ Elderly	+ Family
Initial Units	Crew, Truck, Heli	Crew, Truck, Heli	Crew, Truck
Arrival (Crew/Truck/Heli)	2/4/6	3/5/7	4/6/8

E. User Interface Wireframe

The Pygame interface displays an 8×8 grid with forest (green), normal fire (red), canopy fire (dark red), and civilians (blue). The HUD shows turn, score, available units with deploy buttons, and action buttons for evacuation and ending the turn.

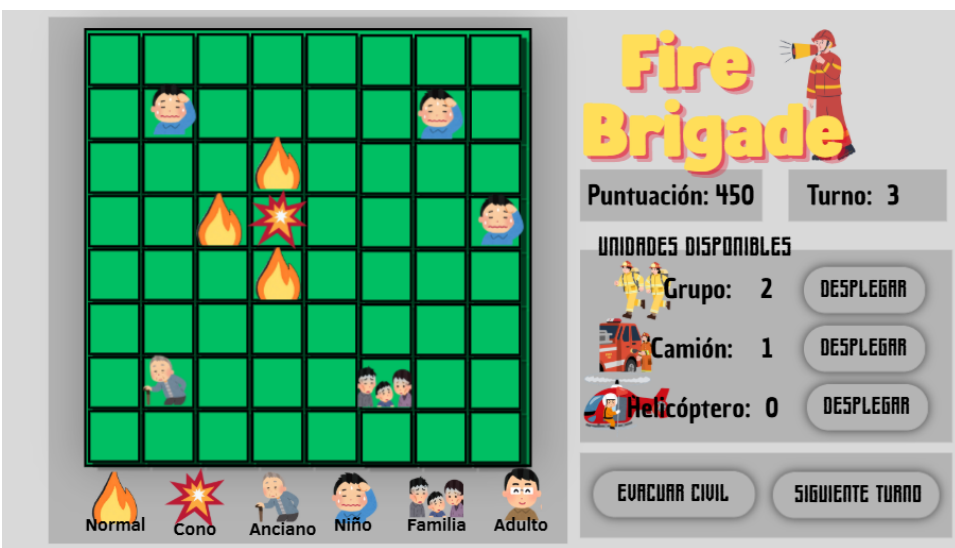

Fig. 4. User Interface Wireframe in Progress

III. PLANNED EXPERIMENTS

Upon implementation completion, the following tests are planned:

- Unit tests for Linked List and BST operations.
- Greedy algorithm validation (highest-risk selection, tie-breaking).
- Backtracking validation (route finding in various fire configurations).
- Integration tests for JSON communication between C++ and Python.
- Gameplay tests for difficulty balance and variant impact.

REFERENCES

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.